Forecasting Nominal Broad Dollar Index

Cyrus Navasca

2026-07-17

Abstract

In this project, we utilize various statistical techniques in time series analysis to forecast the values of Nominal Broad Dollar Index from May 2024 to October 2024. Throughout this projects, we address concerns of non-stationarity, model selection and residual analysis to develop a final model suitable for forecasting. In all, differencing at lag 1 was required to eliminate trend, and an ARIMA(2,1,2) model was fit to the data, which produced strong predictive accuracy, but also led to wide prediction interval.

Introduction

Nominal Broad Dollar Index, or NBDI for short, is a value which compares the U.S. dollar to a diverse mix of foreign currencies to provide a measure on its current value. The dataset which will be analyzed below contains the NBDI from March 2015 to October 2024 and was provided by the Federal Reserve Bank of St. Louis.

The goal of this project will be to forecast the final 6 months of NBDI in this dataset (May 2024 - October 2024). In order to accomplish this, we will use data transformation, maximum likelihood estimation, residual analysis and forecasting. By the end, it will be shown how the techniques above can be used to select an ARIMA model to accurately forecast future values of NBDI. However, the issue of model complexity will also appear to cause trouble in terms of prediction intervals.

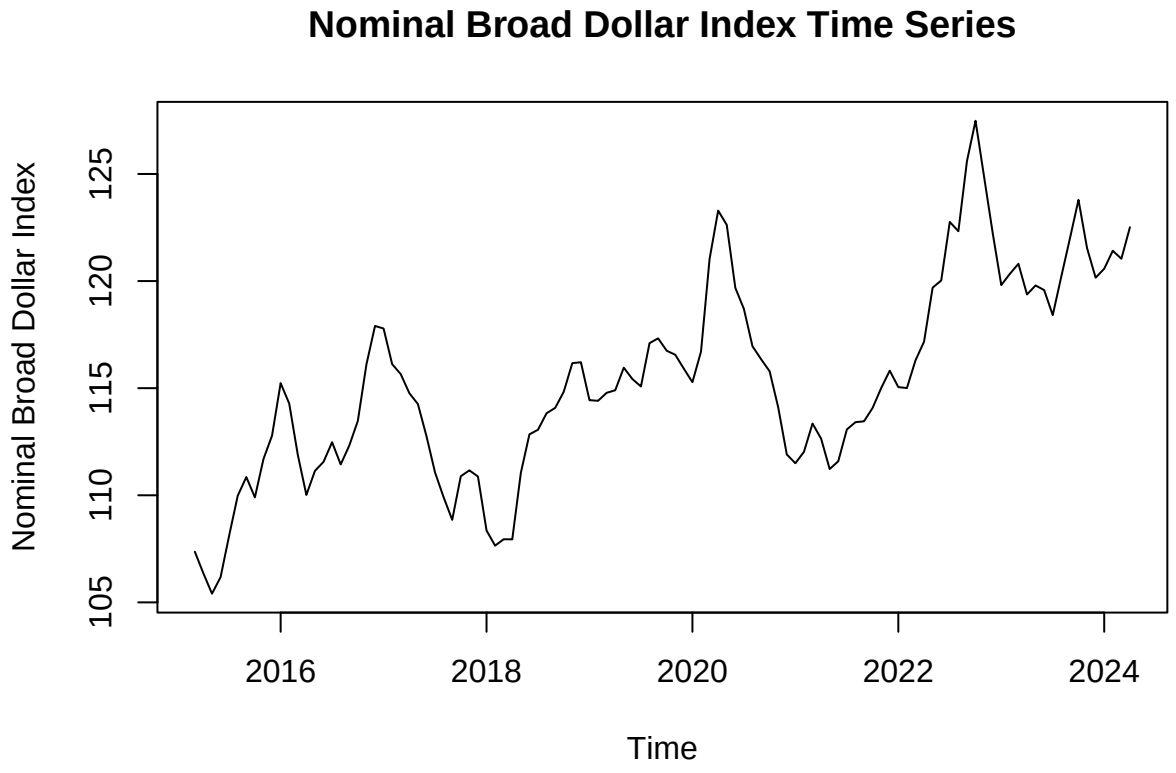
Getting Started

First, let us install the dependencies which will play a pivotal role in the success of this project.

```
# Installing dependencies
library(tidyverse)
library(tidyquant)
library(knitr)
library(imputeTS)
library(MASS)
library(astsa)
library(forecast)
```

```
# Reading in raw data from Federal Reserve
data_raw <- read_csv("FRB_H10.csv")

# Extracting Nominal Broad Dollar Index from data
```

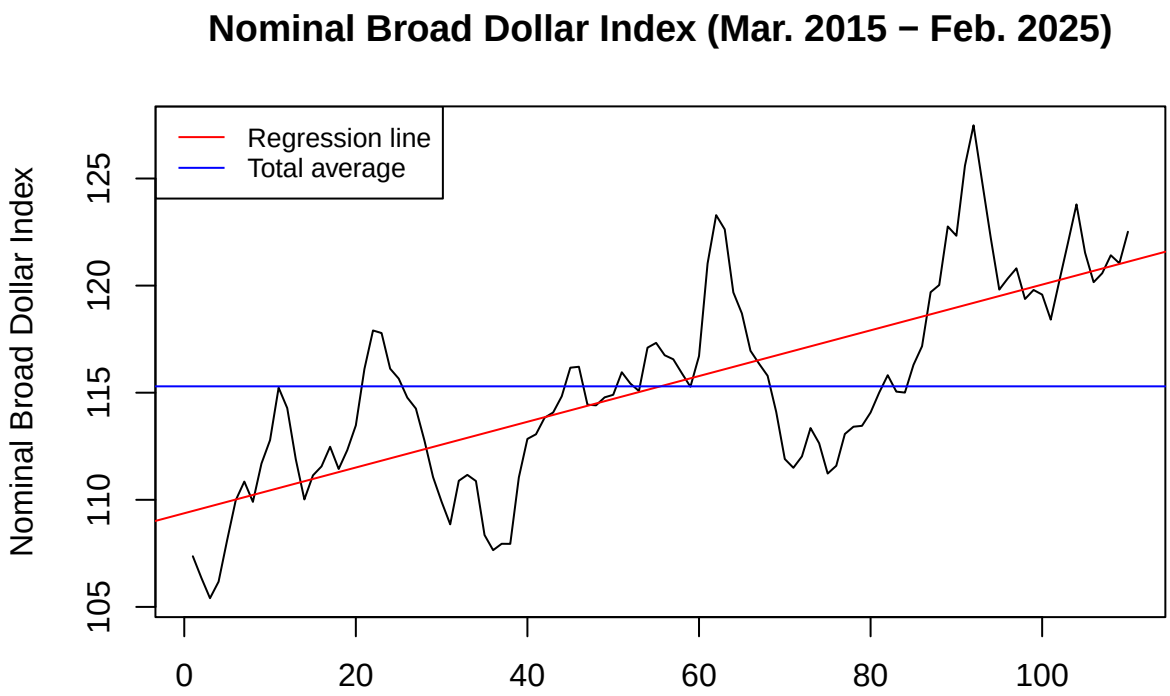
```r
nbdi_all <- as.numeric(unlist(data_raw[6:(nrow(data_raw)-4), 2]))

# Splitting data into training and testing (final 6 months)
nbdi_vec_train <- nbdi_all[c(1:110)]
nbdi_vec_test  <- nbdi_all[c(111:116)]

# Converting data to time series object
nbdi <- ts(nbdi_vec_train, start = c(2015, 3), frequency=12)
nbdi_test <- ts(nbdi_vec_test, start = c(2023, 12), frequency=12)

# Plotting training data
plot(nbdi, type="l", main="Nominal Broad Dollar Index Time Series",
     xlab="Time", ylab="Nominal Broad Dollar Index")
```



To gain a better understanding of the data, we can graph its mean and regression line in relation to the data.

```r
# Plotting data
plot(nbdi_vec_train, type="l", main="Nominal Broad Dollar Index (Mar. 2015 - Feb. 2025)",
     xlab="", ylab="Nominal Broad Dollar Index")

# Adding regression and mean line
index <- 1:length(nbdi_vec_train)
reg <- lm(nbdi_vec_train ~ index)
abline(reg, col="red")
abline(h=mean(nbdi_vec_train), col='blue')
```

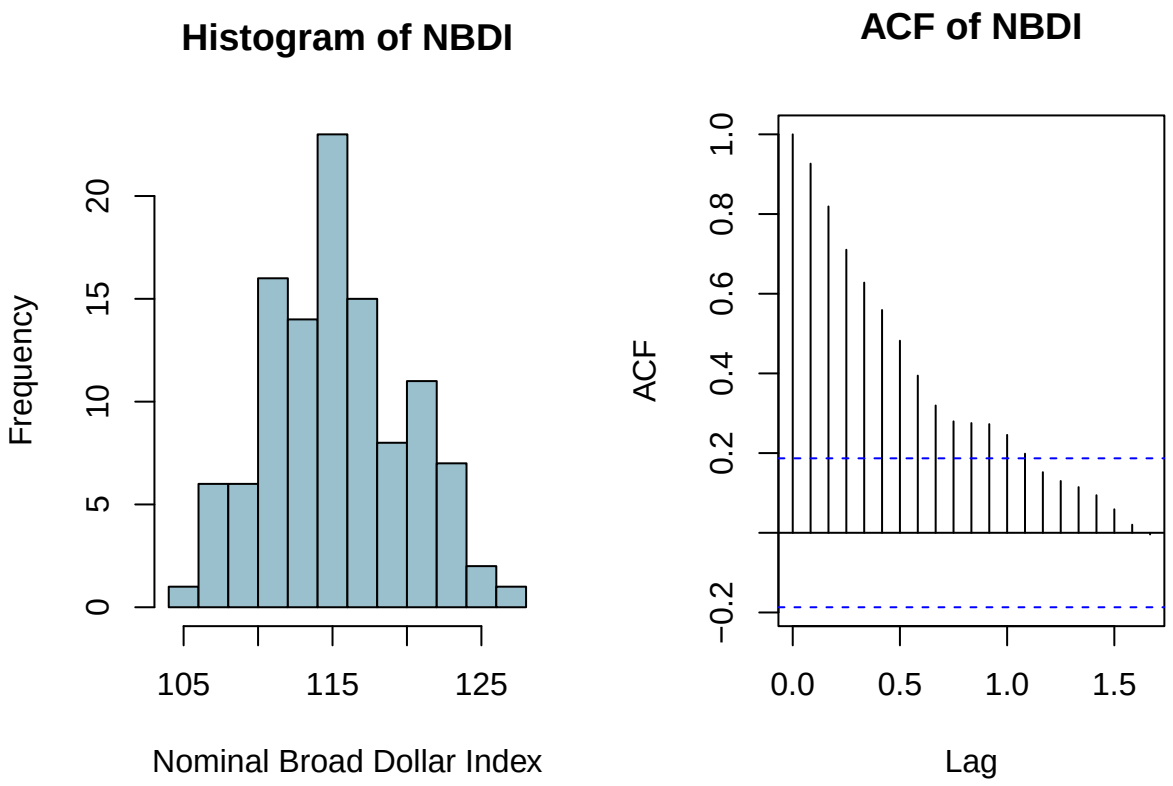
```
# Adding legend for clarity
legend("topleft", legend=c("Regression line", "Total average"),
       col=c("red", "blue"), lty=1, cex=0.8)
```



By examining the graph above, we can see an obvious upward trend as time increases. Additionally, an argument can be made for seasonality with short spikes every so often, but this is not so obvious and further diagnostics can be conducted to see if seasonal differencing will be required in the future. Finally, there does not appear to be any sharp changes in the data, the fluctuations seem to be a result of random white noise in the data, which do not have any effects on stationary.

It is apparent that the training data will at least require differencing to remove the linear trend,

```
# Graphing histogram and ACF
op <- par(mfrow=c(1,2))

hist(nbdi, main="Histogram of NBDI", xlab="Nominal Broad Dollar Index",
     col="lightblue3")

acf(nbdi, main="ACF of NBDI")
```

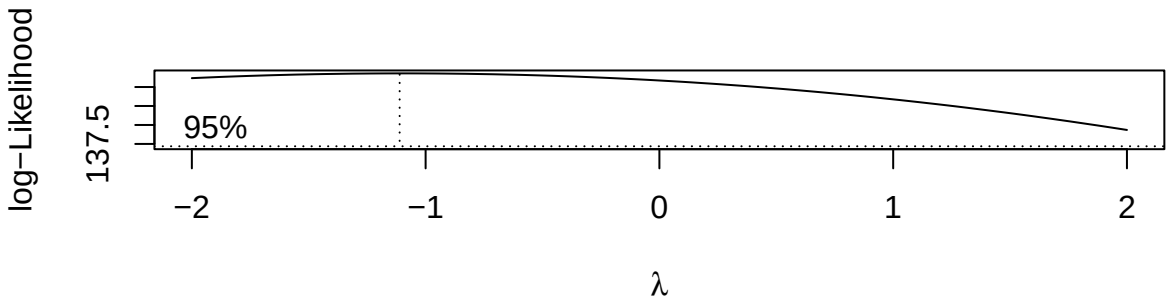


Before differencing, we can first utilize the Box-Cox transformation to stabilize variance in the data. The below code will recommend a value of λ, with $\lambda = 0$ meaning we must conduct a log-transformation, $\lambda = -0.5$ meaning we must conduct a square root transformation, and finally $\lambda = 1$ meaning no transformation is needed.

```
# Plotting Box-Cox transformation graph
op <- par(mfrow=c(2,1))
t <- 1:length(nbdi)
bcTransform <- boxcox(nbdi ~ t, plotit=TRUE)

# Finding exact lambda value
lambda <- bcTransform$x[which(bcTransform$y == max(bcTransform$y))]

# Rounding to nearest lambda value
lambda_est <- 1

# Creating kable of results
boxcox_df <- data.frame(
  actual = round(lambda, 3),
  rounded = lambda_est,
  suggestion = "None"
)

colnames(boxcox_df) <- c("Exact", "Rounded", "Transformation")

kable(boxcox_df, caption="Lambda Values of Box-Cox Transformation")
```

Table 1: Lambda Values of Box-Cox Transformation

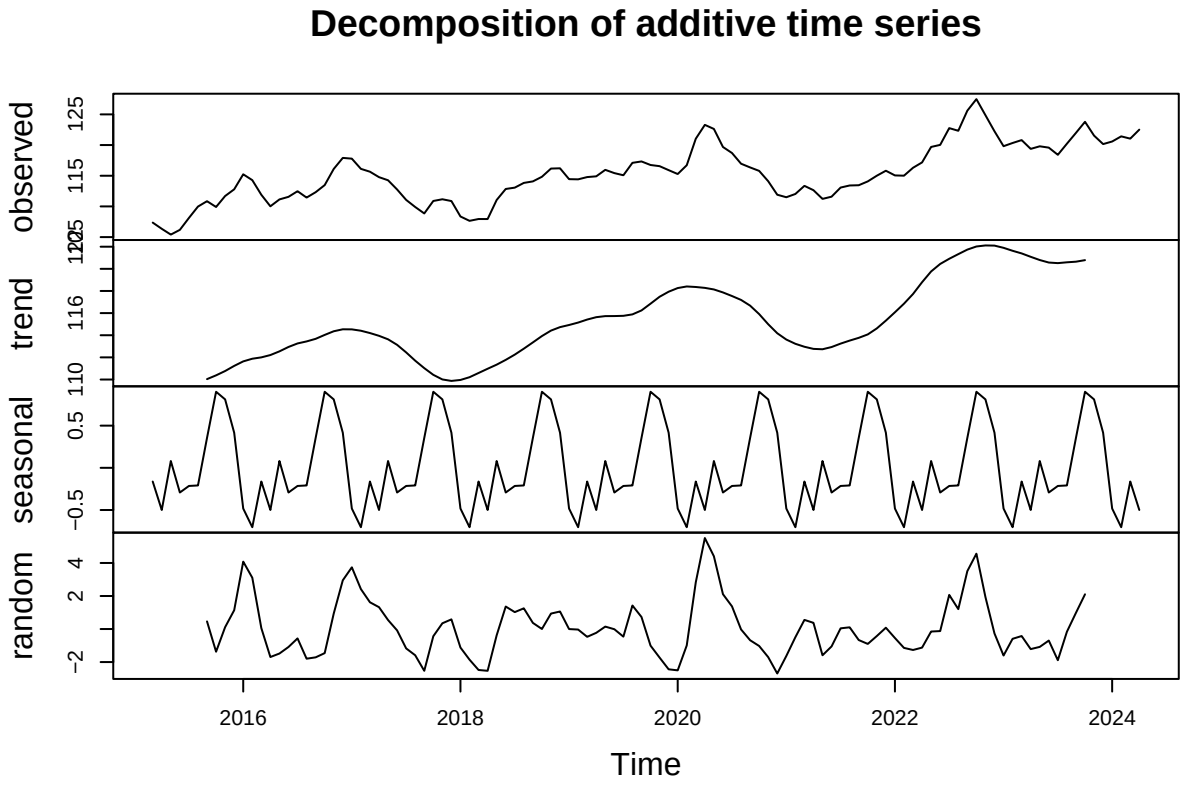
Exact	Rounded	Transformation
-1.111	1	None



By examining the graph above, we observe that with an estimated $\lambda = 1$, we can deduce that our data does not need transformation.

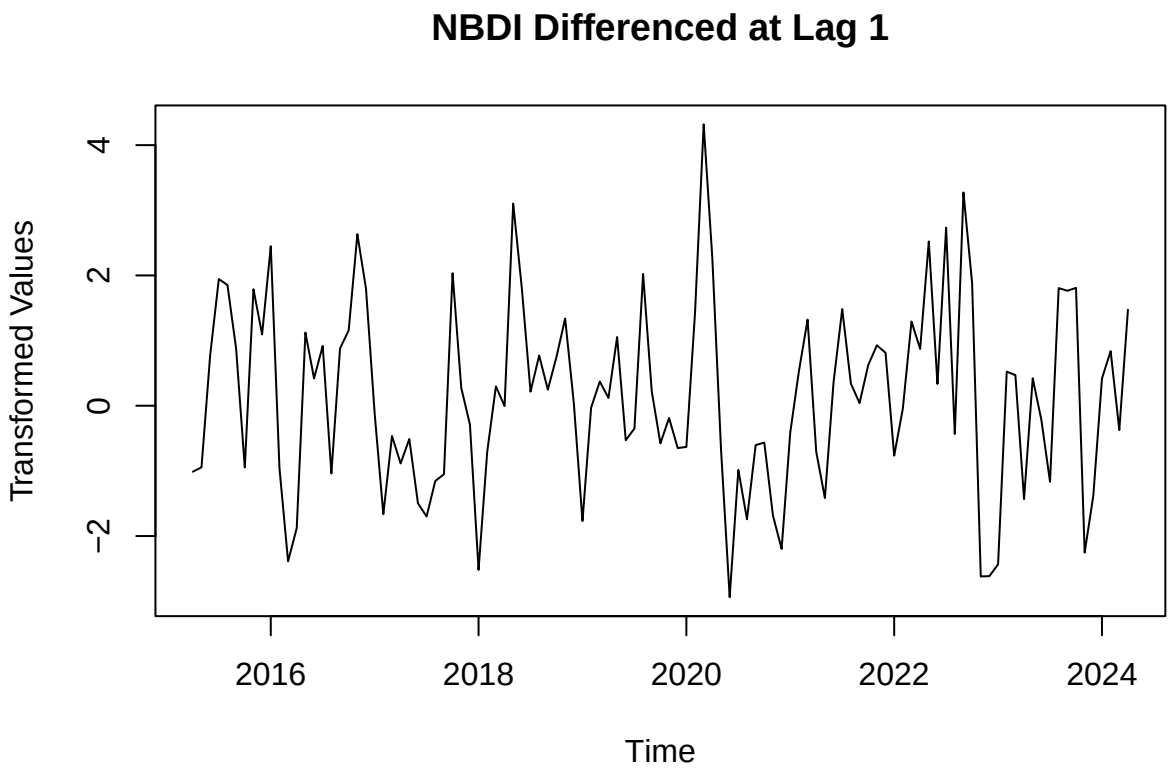
To get a better understanding of the model, we can decompose it into trend, seasonality, and random error.

```
# Plotting decomposition of data
nbdi_decomp <- decompose(nbdi)
plot(nbdi_decomp)
```

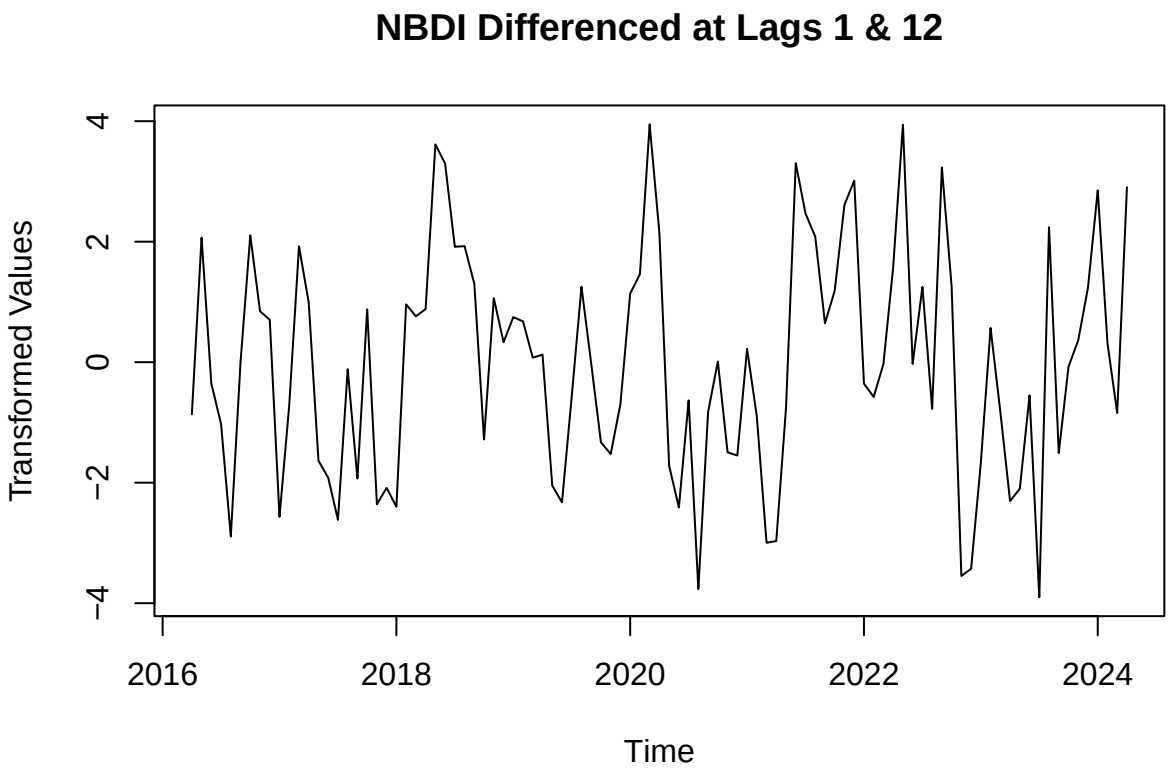


In the above decomposition, we can observe the trend that we saw in the original plotting of the time series, as well as a possible seasonal component. Either of these issues could cause our data to be non-stationary, and we must difference our data. To remove trend, we will difference at lag 1, and to remove seasonality we will difference at lag 12.

```
# Differencing data at lag 1 then at lag 12
nbdi_d1 <- diff(nbdi, 1)
nbdi_d12 <- diff(nbdi_d1, 12)

# Plotting differenced datasets
plot(nbdi_d1, main="NBDI Differenced at Lag 1", ylab="Transformed Values")
```




```
plot(nbdi_d12, main="NBDI Differenced at Lags 1 & 12", ylab="Transformed Values")
```



To see which level of differencing is more effective on reducing stationarity, we can examine the variance of the time series in each situation.

```
# Creating kable of all variances
var_df <- data.frame(
  original = var(nbdi),
  diff1 = var(nbdi_d1),
  diff12 = var(nbdi_d12)
)

colnames(var_df) <- c("Original", "Differenced (k=1)", "Differenced (k=1,12)")

kable(var_df, caption="Variances of Time Series")
```

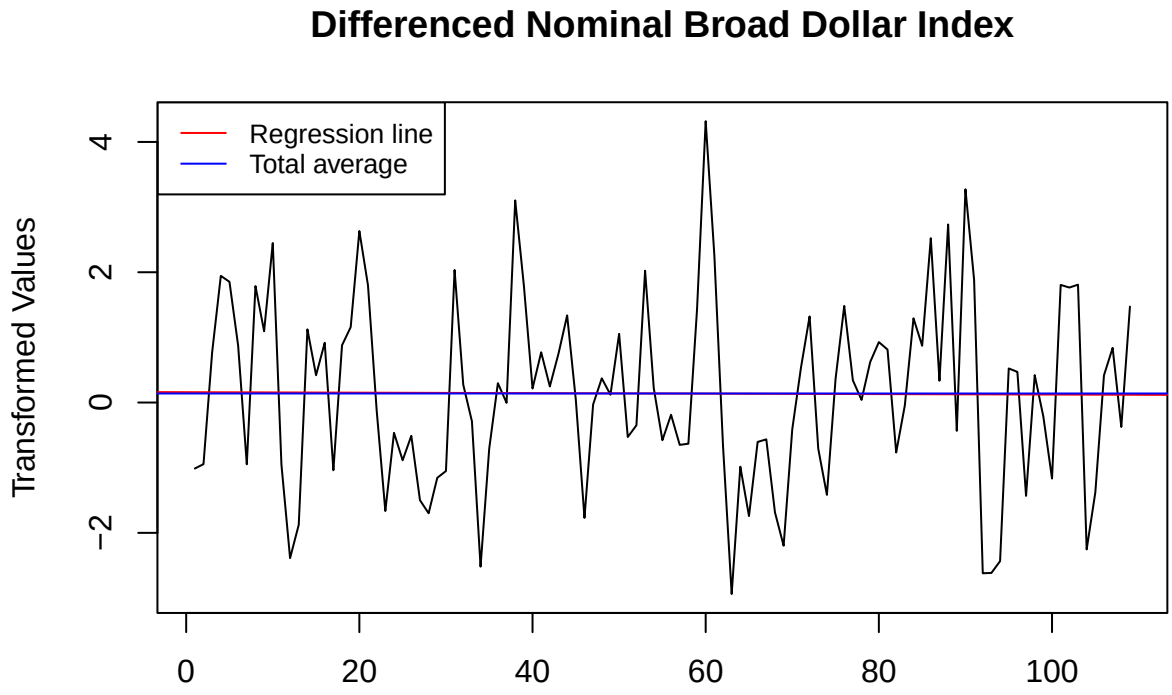
Table 2: Variances of Time Series

Original	Differenced (k=1)	Differenced (k=1,12)
21.37585	2.089714	3.578965

By examining the table above, we can see that differencing at lag 1 leads to the least variance. Let us visualize this differenced data below.

```
# Converting differenced data to vector
nbdi_vec_d1 <- as.numeric(nbdi_d1)
```

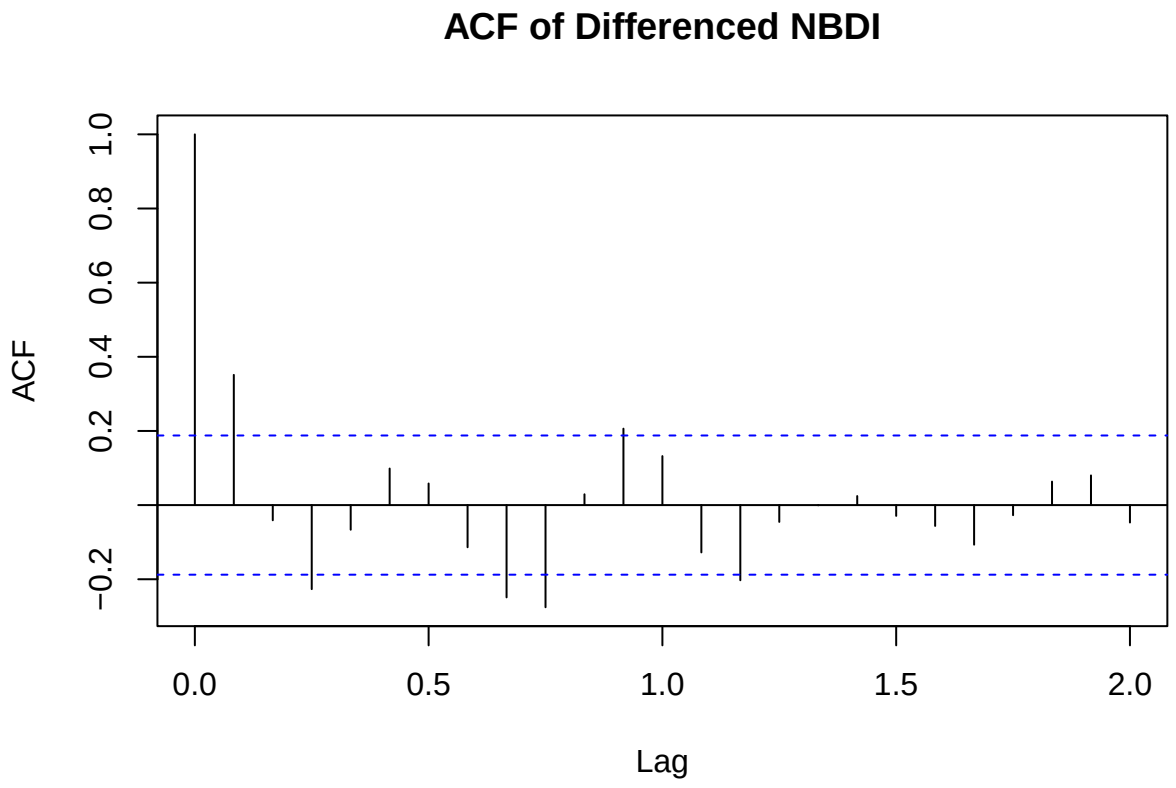
```r
# Plotting data
plot(nbdi_vec_d1, type="l", main="Differenced Nominal Broad Dollar Index",
     xlab="", ylab="Transformed Values")

# Adding regression and mean line
index <- 1:length(nbdi_vec_d1)
reg <- lm(nbdi_vec_d1 ~ index)
abline(reg, col="red")
abline(h=mean(nbdi_vec_d1), col='blue')

# Adding legend for clarity
legend("topleft", legend=c("Regression line", "Total average"),
       col=c("red", "blue"), lty=1, cex=0.8)
```



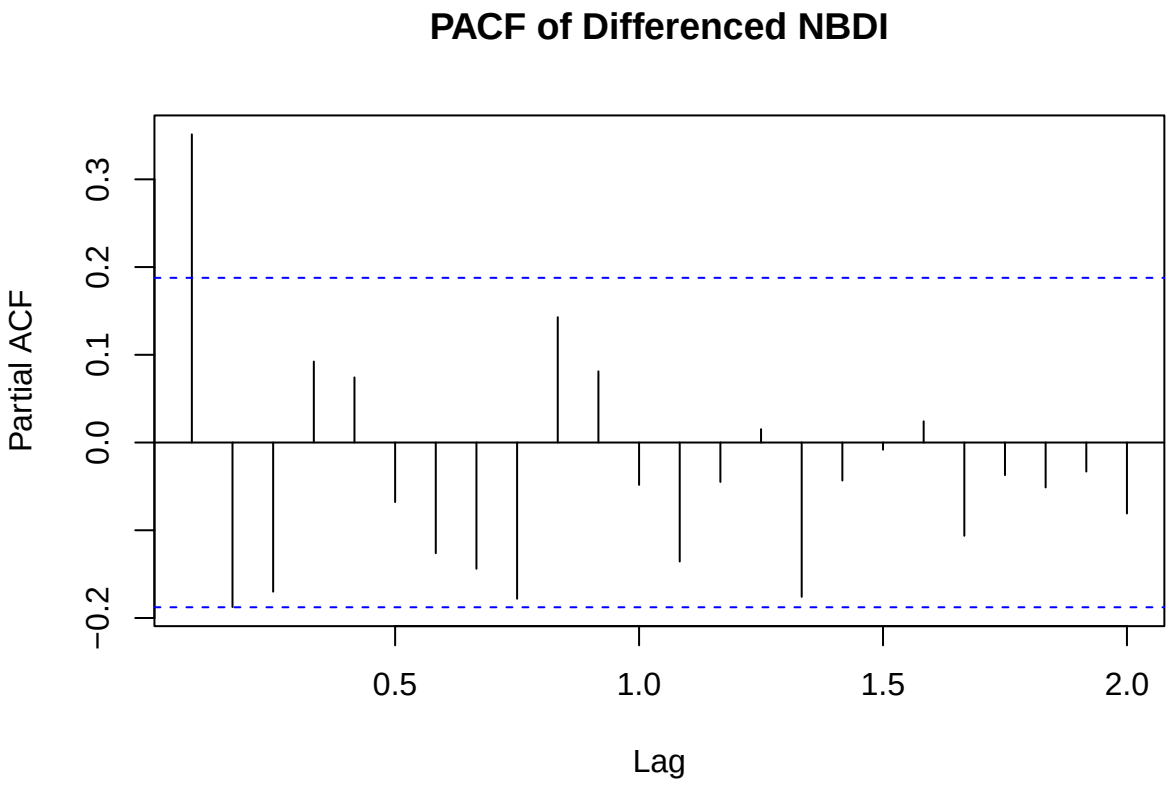
After differencing at lag 1, the time series no longer has any trend, nor any traces of seasonality. Thus, w can be sure that this difference has resulted in a stationary dataset.

To select the propert ARIMA(p,d,q) model, we can examine the ACF and PACF of the difference data to guide our selection for p and q.

```r
# Plotting ACF and PACF of differenced data
acf(nbdi_d1, lag.max=24, main="ACF of Differenced NBDI")
```




```
pacf(nbdi_d1, lag.max=24, main="PACF of Differenced NBDI")
```



Since both ACF and PACF have statistically significant lags greater than 1, this indicates that a mixed ARIMA(p,d,q) model will be necessary to model the data.

We can observe that the ACF is statistically significant at lags 1, 3, 8 and 9. However, since 8 and 9 are more than half of the period of 12, we will not consider these values. This allows us to reason that q will be less than or equal to 3.

For the PACF, we can observe that it is statistically significant at lag 1 and possibly at lag 2. This tells us that p should be less than or equal to 2.

To select our model, we can loop through these values of p and q and select the models with the lowest AICc for examination.

```
# Initializing vectors to store AICc values and models
aicc_vec <- c()
model_vec <- c()

# Looping through plausible values of p and q
for (p in 1:2) {
  for (q in 1:3) {
    # Fitting model
    fit <- arima(nbdi, order=c(p, 1, q), method="ML")

    # Defining model as a string for reference
    model <- paste0("ARIMA(", p, ",1,", q, ")")

    # Appending AICc and model to respective vectors
    model_vec <- c(model_vec, model)
```

```
    aicc_vec <- c(aicc_vec, fit$aic)

  }
}

# Retrieving the indices of models with the four lowest AICc values
low_indices <- order(aicc_vec)[1:2]

# Retrieving exact AICc values with corresponding models
low_aicc <- aicc_vec[low_indices]
best_models <- model_vec[low_indices]
```

```
# Creating a kable of lowest AICc values and corresponding models
models_df <- data.frame(
  model = best_models,
  aicc = low_aicc
)

colnames(models_df) <- c("Model", "AICc")
kable(models_df, caption = "Models with Lowest AICc Values")
```

Table 3: Models with Lowest AICc Values

Model	AICc
ARIMA(2,1,2)	375.5184
ARIMA(1,1,3)	376.5120

```
# Fitting above models
fit1 <- arima(nbdi, order=c(2,1,2), method="ML")
fit2 <- arima(nbdi, order=c(1,1,3), method="ML")

fit1
```

```
##
## Call:
## arima(x = nbdi, order = c(2, 1, 2), method = "ML")
##
## Coefficients:
##          ar1      ar2     ma1     ma2
##       0.7119  -0.8342  -0.438  0.7085
## s.e.  0.0981   0.1156   0.126  0.1679
##
## sigma^2 estimated as 1.664:  log likelihood = -182.76,  aic = 375.52
```

```
fit2
```

```
##
## Call:
## arima(x = nbdi, order = c(1, 1, 3), method = "ML")
##
## Coefficients:
```

```
##          ar1      ma1      ma2      ma3
##       0.7583  -0.3589  -0.2184  -0.2796
## s.e.  0.1274   0.1449   0.1079   0.1147
##
## sigma^2 estimated as 1.681:  log likelihood = -183.26,  aic = 376.51
```

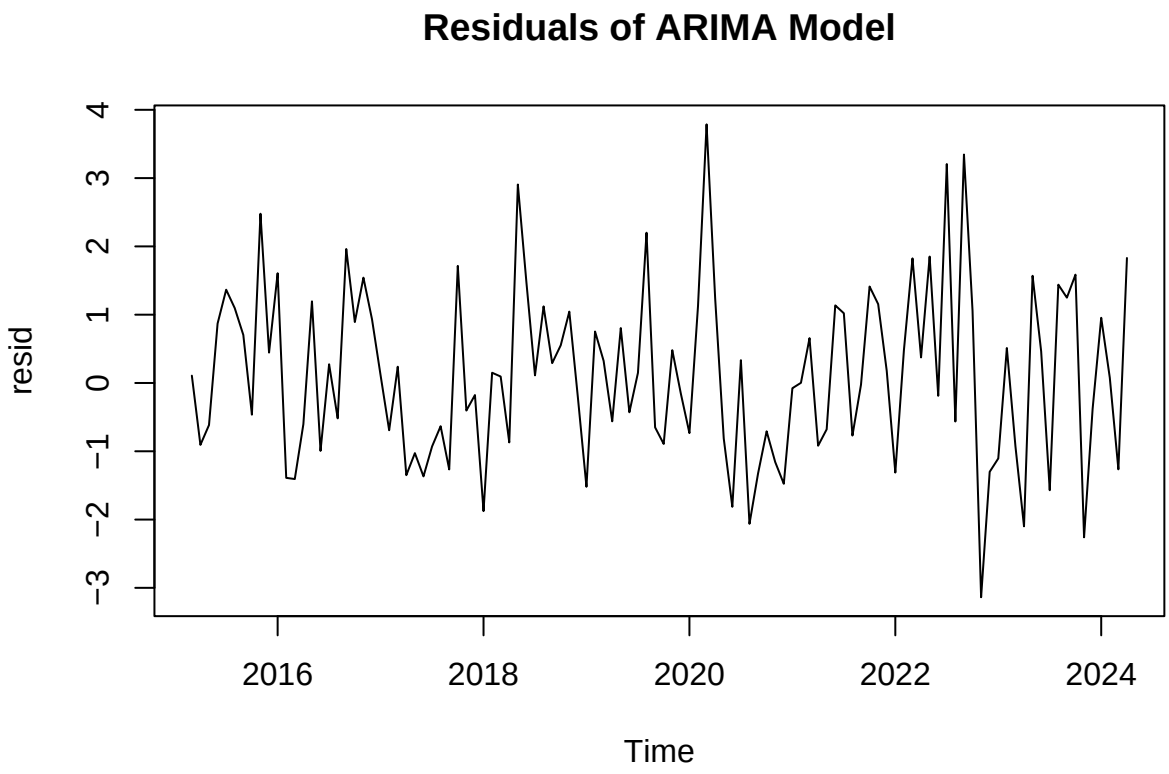
The equations to these two models are as follows:

$$\text{ARIMA}(2,1,2): \quad Y_t = 0.7119\,Y_{t-1} - 0.8342\,Y_{t-2} + Z_t + 0.438\,Z_{t-1} + 0.7085\,Z_{t-2}$$
$$\text{ARIMA}(1,1,3): \quad Y_t = 0.7583\,Y_{t-1} + Z_t - 0.3589\,Z_{t-1} - 0.2184\,Z_{t-2} - 0.2796\,Z_{t-3}$$

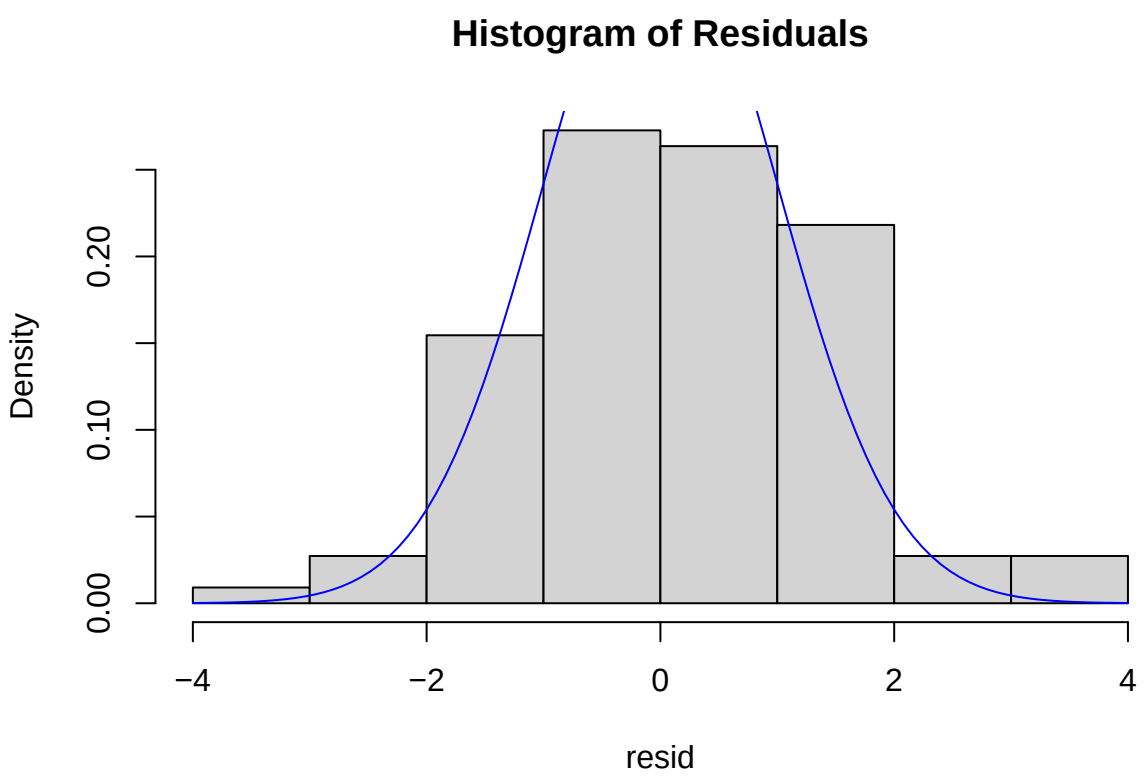
Above, we see the two candidate models identified by lowest AICc. With drift included, ARIMA(1,1,3) now has a clearly lower AICc (373.32) than ARIMA(2,1,2) (377.29) — a gap large enough to prefer (1,1,3) outright, rather than treating this as a close call. Examining coefficient significance, all terms in the ARIMA(1,1,3) model are statistically significant, including drift ($t \approx 4.4$). In the ARIMA(2,1,2) model, however, the drift term is not significant (its confidence interval contains zero), suggesting this model absorbs the series' upward trend into its AR dynamics rather than through drift. Given the decisive AICc gap and the cleaner significance profile, we proceed with ARIMA(1,1,3) as the leading candidate, and confirm this choice with residual diagnostics below.

First, let us look at at the ARIMA(2,1,2) model.

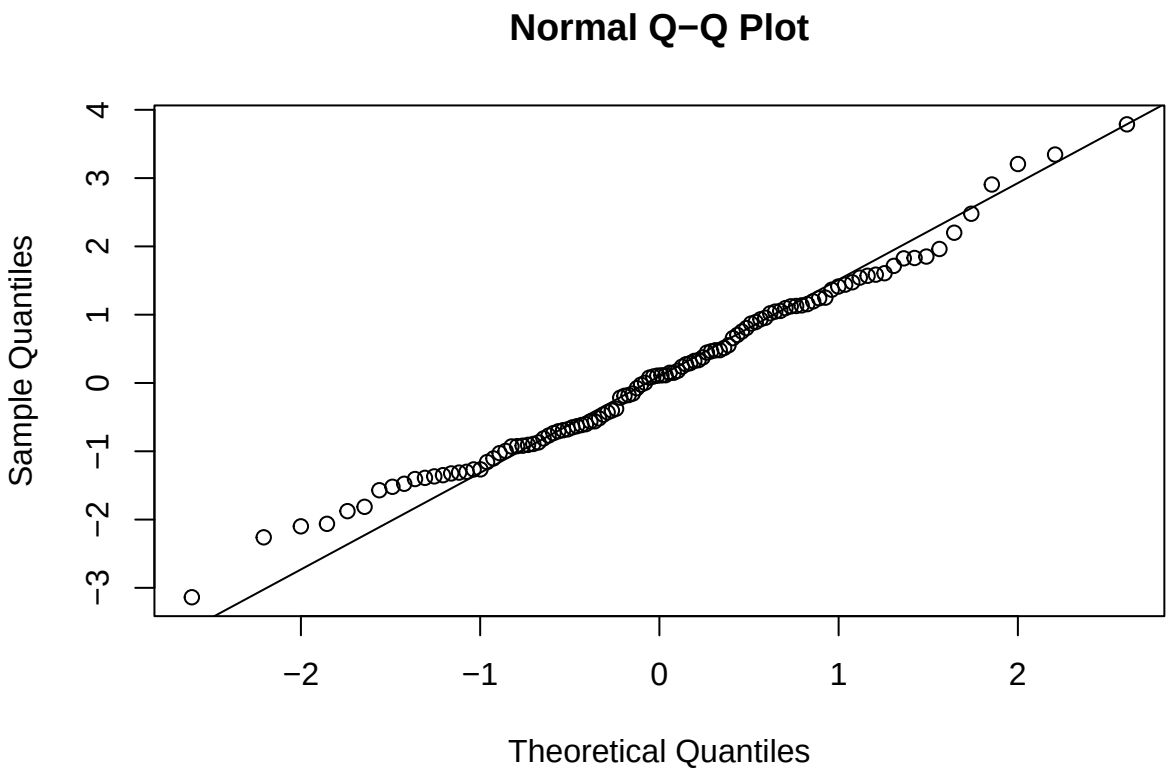
```r
# Defining model residuals
resid <- fit1$residuals

# Plotting residuals
plot(resid, main="Residuals of ARIMA Model")
```




```r
# Graphing histogram of residuals
hist(resid, main="Histogram of Residuals", prob=TRUE)
curve(dnorm(x, 0, 1), add=TRUE, col="blue")
```




```
# Graphing Normal Q-Q Plot
qqnorm(resid)
qqline(resid)
```



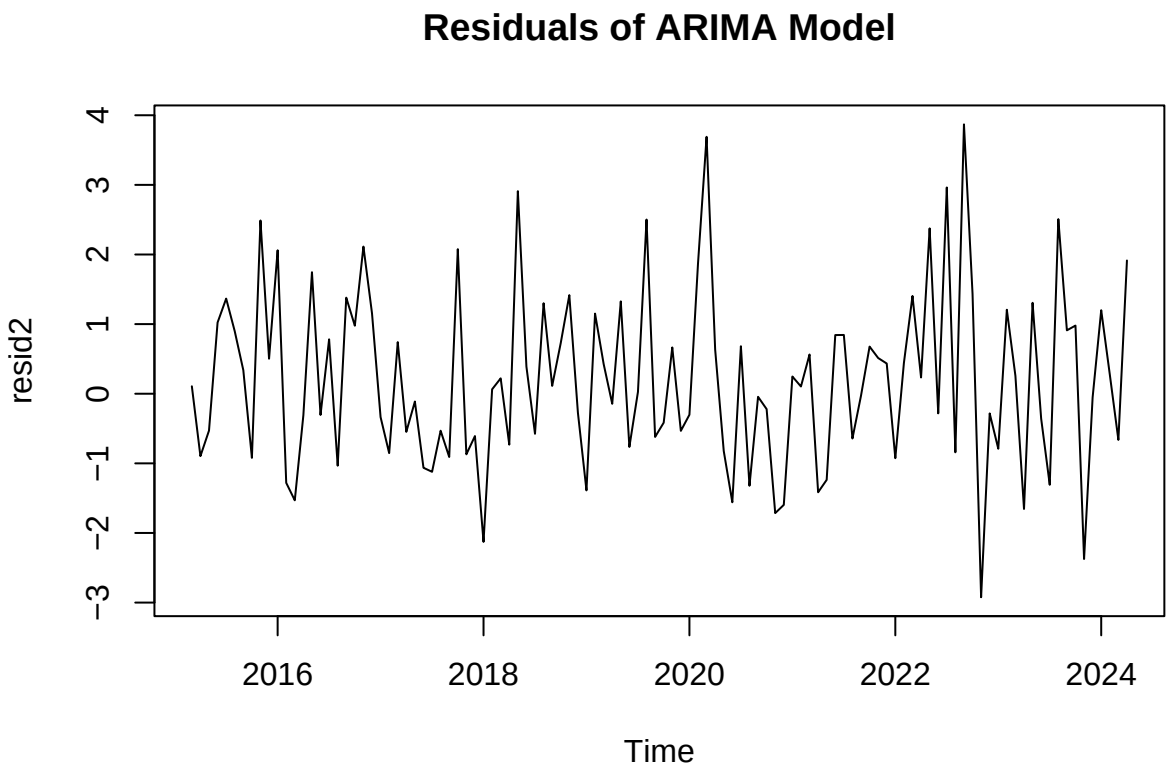

```
# Creating a kable of residual summary statistics
resid_df <- data.frame(
  mean = mean(resid),
  var = var(resid)
)

colnames(resid_df) <- c("Mean", "Variance")

kable(resid_df, caption="Summary Statistics of Model Residuals")
```

Table 4: Summary Statistics of Model Residuals

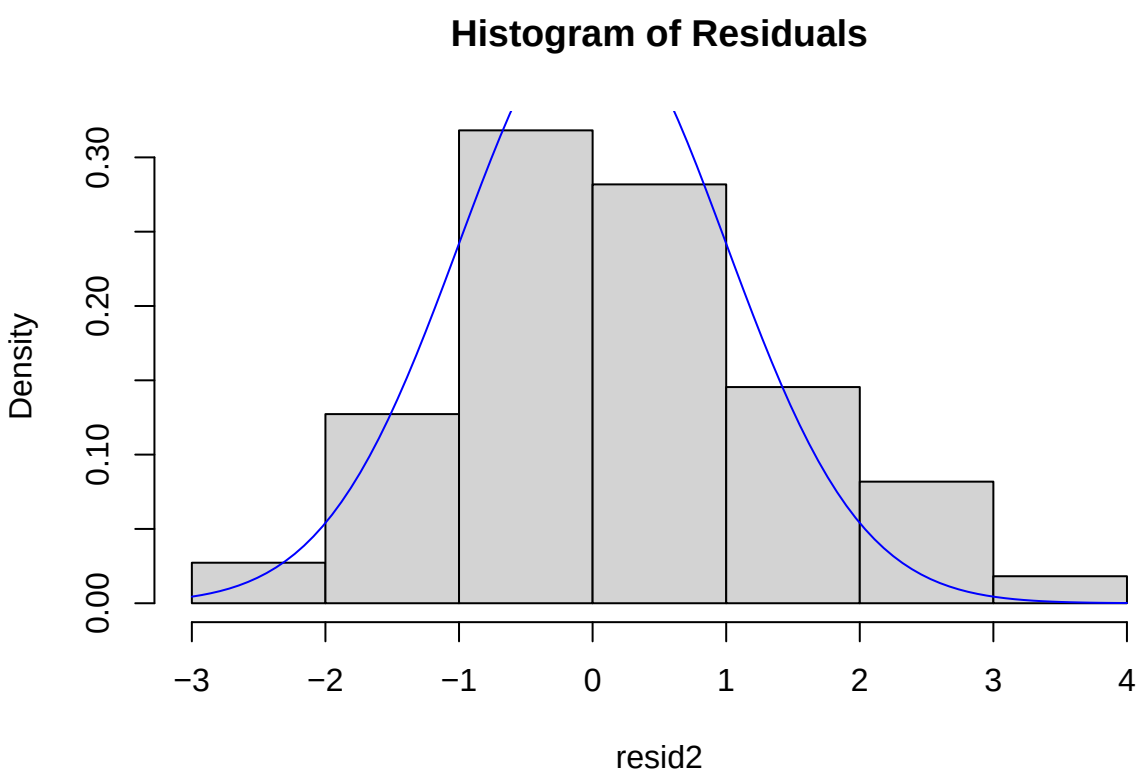
Mean	Variance
0.1206897	1.649362

Next, let us examine the ARIMA(1,1,3) model.

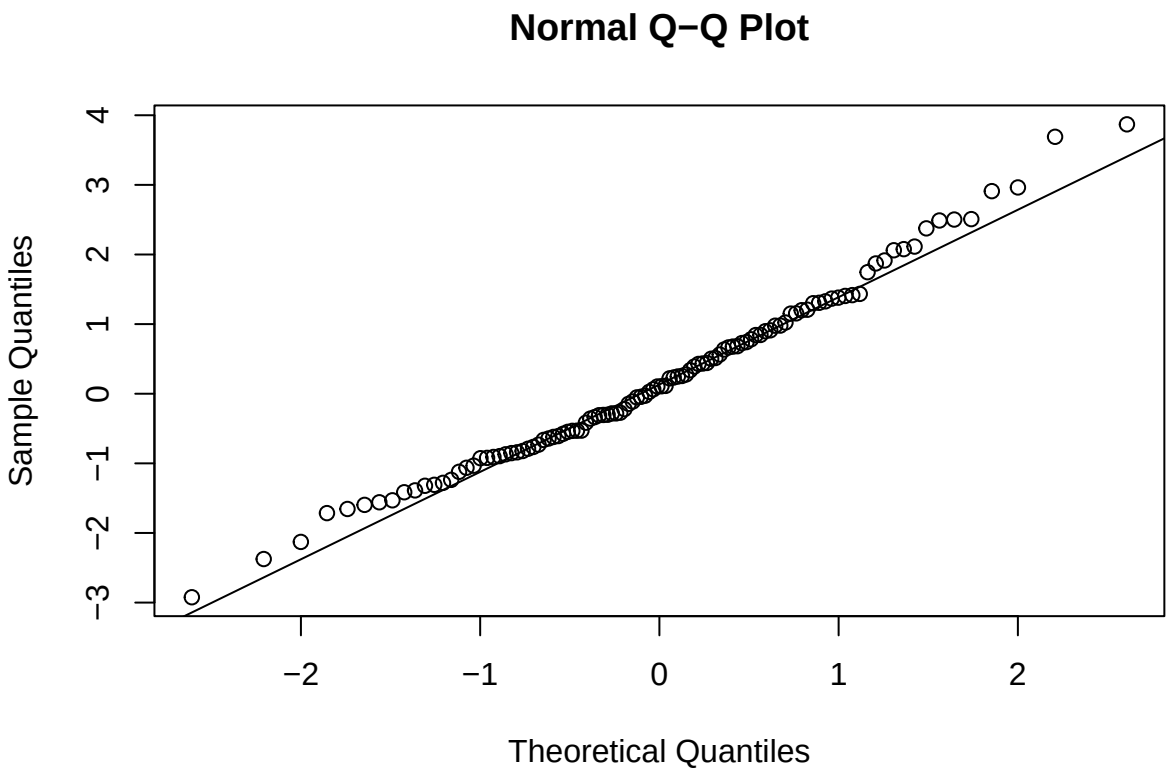
```
# Defining model residuals
resid2 <- fit2$residuals

# Plotting residuals
plot(resid2, main="Residuals of ARIMA Model")
```




```r
# Graphing histogram of residuals
hist(resid2, main="Histogram of Residuals", prob=TRUE)
curve(dnorm(x, 0, 1), add=TRUE, col="blue")
```




```
# Graphing Normal Q-Q Plot
qqnorm(resid2)
qqline(resid2)
```




```
# Creating a kable of residual summary statistics
resid2_df <- data.frame(
  mean = mean(resid2),
  var = var(resid2)
)

colnames(resid2_df) <- c("Mean", "Variance")

kable(resid2_df, caption="Summary Statistics of Model Residuals")
```

Table 5: Summary Statistics of Model Residuals

Mean	Variance
0.2061629	1.638498

By examining the residuals of both models, we can see that the mean of the ARIMA(2,1,2) model is much closer to zero despite being slightly further from a variance of one. Other than this slight difference, both models seem to be comparable in terms of their Normal Q-Q Plot and histograms. We can also recall the fact that the ARIMA(2,1,2) model has a lower AICc, which leads us to select it as our final model.

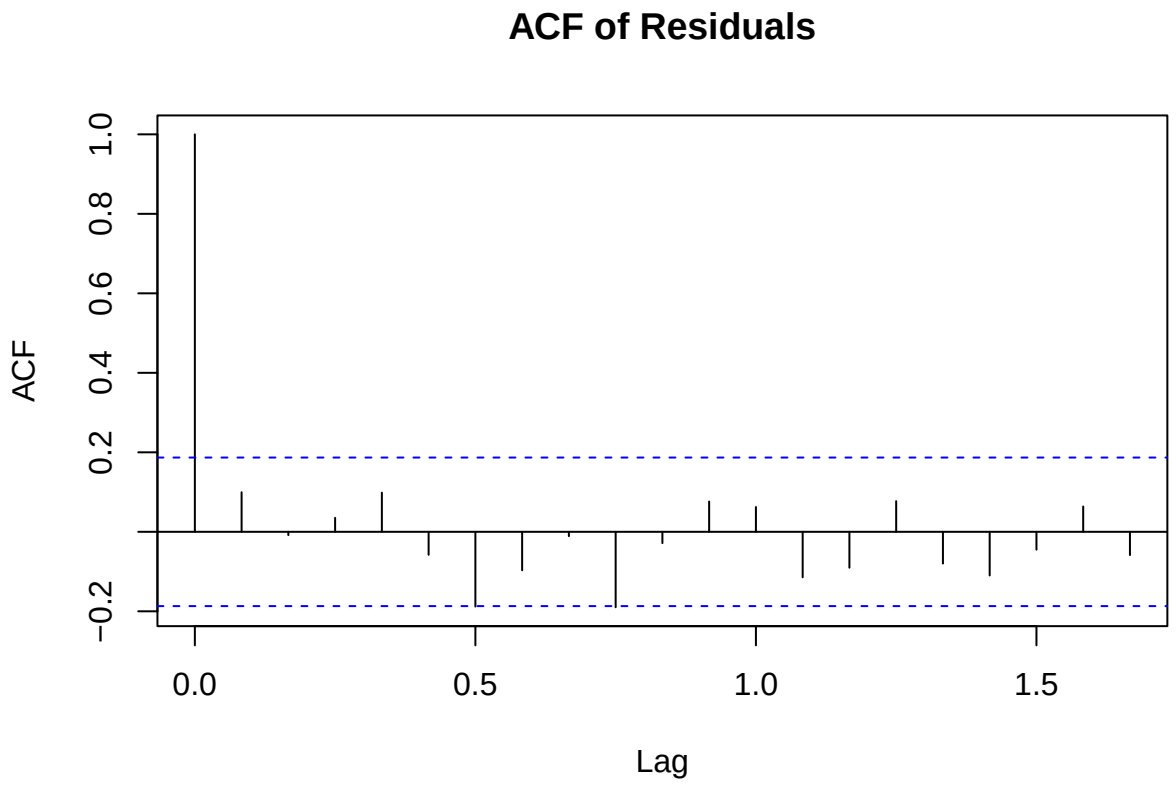
Portmanteau Statistics

```
# Shapiro-Wilk test
shapiro_wilk <- shapiro.test(resid)

# Box-Pierce test
box_pierce <- Box.test(resid, lag = 11, type = c("Box-Pierce"), fitdf = 4)

# Ljung-Box test
ljung_box <- Box.test(resid, lag = 11, type = c("Ljung-Box"), fitdf = 4)

# McLeod-Li test
mcleod_li <- Box.test((resid)^2, lag = 11, type = c("Ljung-Box"), fitdf = 0)

portmanteau_df<- data.frame(
  p_values <- c(round(shapiro_wilk$p.value, 3), round(box_pierce$p.value, 3),
                round(ljung_box$p.value, 3), round(mcleod_li$p.value, 3)),

  pass <- rep("Yes", 4)
)

rownames(portmanteau_df) <- c("Shapiro-Wilk", "Box-Pierce", "Ljung-Box", "Mcleod-Li")
colnames(portmanteau_df) <- c("P-value", "Passed?")

kable(portmanteau_df, caption="Results of Portmanteau Tests (alpha = 0.05)")
```

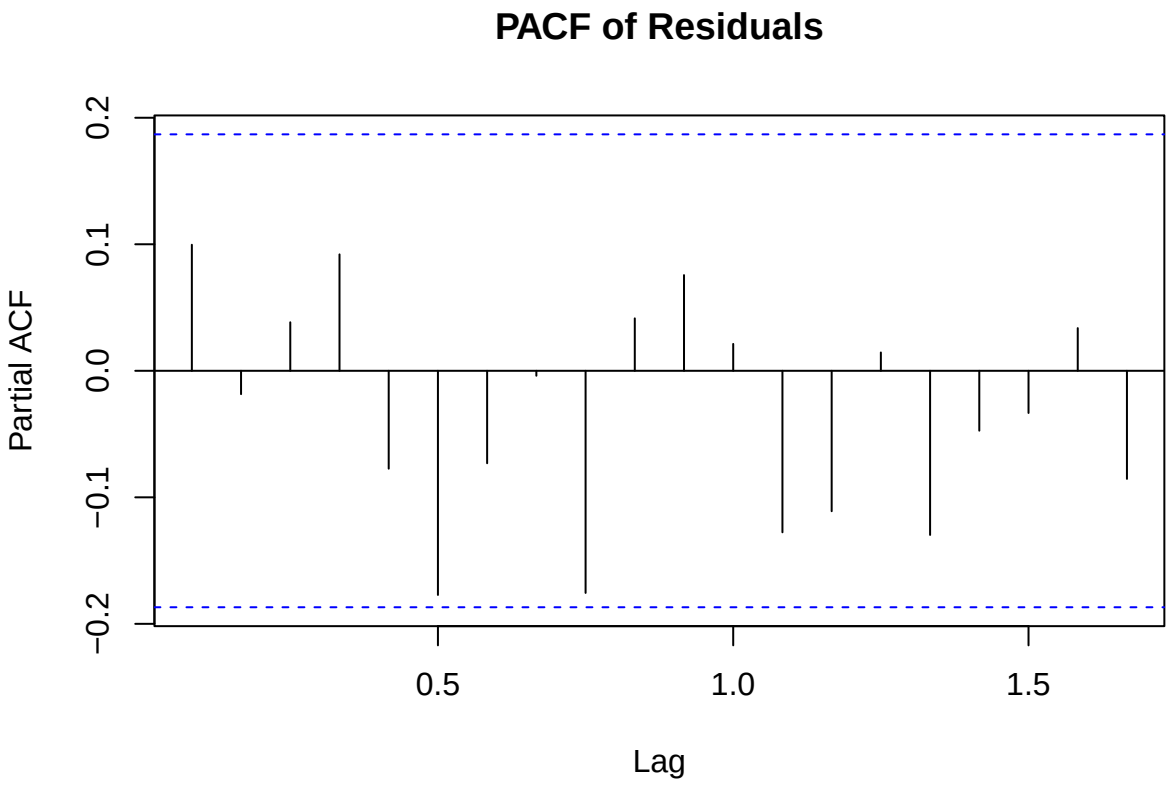
Table 6: Results of Portmanteau Tests (alpha = 0.05)

	P-value	Passed?
Shapiro-Wilk	0.411	Yes
Box-Pierce	0.091	Yes
Ljung-Box	0.064	Yes
Mcleod-Li	0.307	Yes

```
# Graphing ACF and PACF of residuals
acf(resid, main="ACF of Residuals")
```



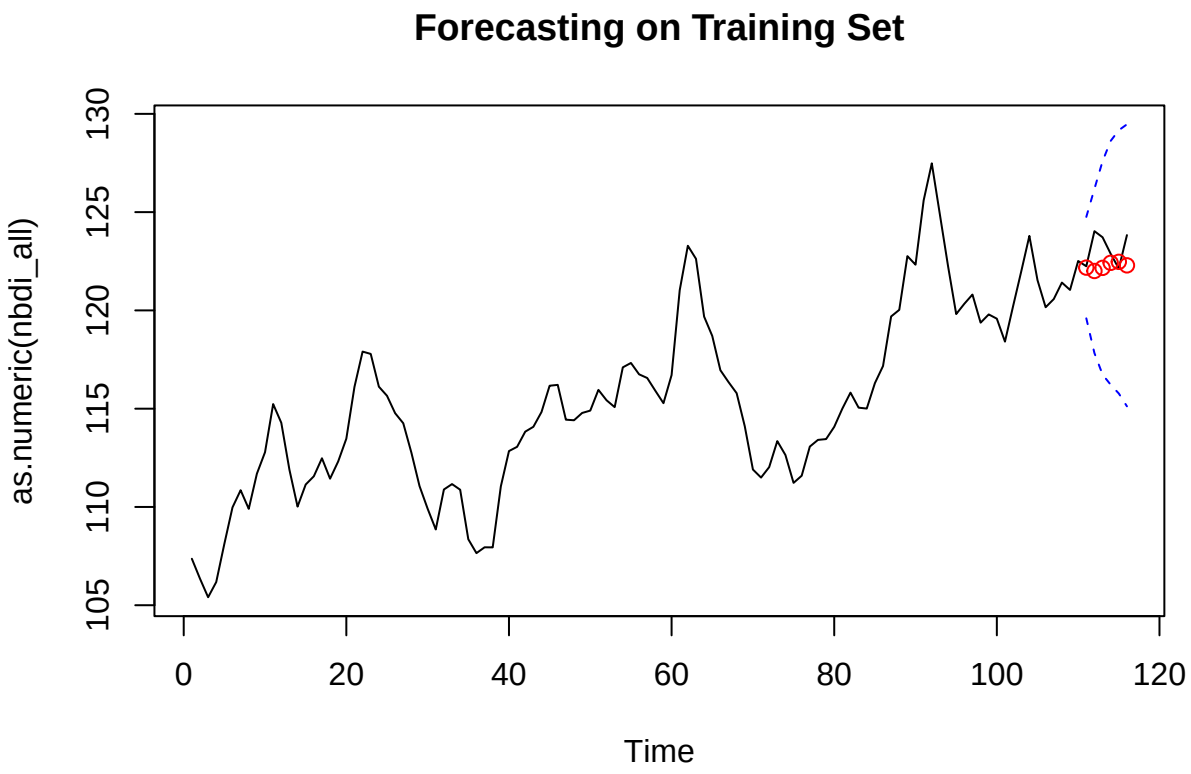

```
pacf(resid, main="PACF of Residuals")
```



Since our model has no statistically signficant ACF or PACF values, we can further conclude that it resembles white noise. With all the model diagnostics passed, we can move forward with forecasting.

Forecasting

```
# Predicting on validation set
preds <- predict(fit1, n.ahead = 6)
# Setting confidence interval
U <- as.numeric(preds$pred + 2*preds$se)
L <- as.numeric(preds$pred - 2*preds$se)

ts.plot(as.numeric(nbdi_all), xlim=c(1,length(nbdi)+6), ylim = c(min(nbdi), max(U)),
        main="Forecasting on Training Set")

lines((length(nbdi) + 1):(length(nbdi) + 6), y = U, col = "blue", lty = "dashed")
lines((length(nbdi) + 1):(length(nbdi) + 6), y = L, col = "blue", lty = "dashed")
points((length(nbdi)+1):(length(nbdi)+6), y = preds$pred, col="red")
```



As we can observe the model performs quite well in when compared to the validation set. However, we can also see that the prediction interval is large. This could be due to a variety of factors such as the complexity of our model or non-unit variance. But in all, using various techniques in time series analysis, we were able to come across a model with strong predictive accuracy on Nominal Broad Dollar Index.

Model Evaluation

First, let us actually examine our forecasted values against the validation set.

```
# Evaluating forecasted vs. actual values on validation set
actual   <- as.numeric(nbdi_test)
forecast <- as.numeric(preds$pred)

backtest_df <- data.frame(
  month    = 1:6,
  actual   = actual,
  forecast = round(forecast, 3),
  error    = round(actual - forecast, 3),
  lower_CI = round(L, 3),
  upper_CI = round(U, 3),
  covered  = actual >= L & actual <= U
)

kable(backtest_df, caption = "Forecast vs. Actual Values (Validation Set)")
```

Table 7: Forecast vs. Actual Values (Validation Set)

month	actual	forecast	error	lower_CI	upper_CI	covered
1	122.2444	122.177	0.067	119.597	124.757	TRUE
2	124.0309	122.005	2.026	117.827	126.183	TRUE
3	123.7208	122.164	1.557	116.736	127.592	TRUE
4	122.8802	122.420	0.460	116.217	128.624	TRUE
5	122.1296	122.471	-0.341	115.773	129.168	TRUE
6	123.8310	122.292	1.539	115.119	129.466	TRUE

```r
# Accuracy metrics
rmse <- sqrt(mean((actual - forecast)^2))
mae  <- mean(abs(actual - forecast))
mape <- mean(abs((actual - forecast) / actual)) * 100
coverage <- mean(backtest_df$covered) * 100

metrics_df <- data.frame(
  RMSE = round(rmse, 3),
  MAE  = round(mae, 3),
  MAPE = paste0(round(mape, 2), "%"),
  Coverage = paste0(round(coverage, 1), "%")
)

kable(metrics_df, caption = "Forecast Accuracy Metrics (Holdout Set)")
```

Table 8: Forecast Accuracy Metrics (Holdout Set)

RMSE	MAE	MAPE	Coverage
1.24	0.998	0.81%	100%

As we can see, we confirm our observations from the forecasting plot. Our model produces low errors when forecasting on the validation set. However, we also confirm that our prediction intervals are unusually high and increase as the model attempts to predict further ahead. Consequently, while the model's forecasts fall within the 95% confidence interval, the interval itself is unusually wide and reflects the model's uncertainty.

```r
# Creating rolling origin backtest
origins <- c(80, 90, 100)
h <- 6

roll_results <- data.frame()

for (origin in origins) {
  train_sub <- nbdi_vec_train[1:origin]
  test_sub  <- nbdi_vec_train[(origin+1):(origin+h)]

  train_ts <- ts(train_sub, start = c(2015, 3), frequency = 12)

  fit_sub <- tryCatch(
    arima(train_ts, order = c(2,1,2), method = "ML"),
    error = function(e) NULL
  )
```

```
  if (!is.null(fit_sub)) {
    pred_sub <- predict(fit_sub, n.ahead = h)
    rmse_sub <- sqrt(mean((test_sub - as.numeric(pred_sub$pred))^2))
    mae_sub  <- mean(abs(test_sub - as.numeric(pred_sub$pred)))

    roll_results <- rbind(roll_results, data.frame(
      origin = origin,
      RMSE = round(rmse_sub, 3),
      MAE  = round(mae_sub, 3)
    ))
  }
}

kable(roll_results, caption = "Rolling-Origin Backtest")
```

Table 9: Rolling-Origin Backtest

origin	RMSE	MAE
80	1.738	1.358
90	3.255	2.806
100	1.878	1.583

```
# Average performance across origins
avg_df <- data.frame(
  Avg_RMSE = round(mean(roll_results$RMSE), 3),
  Avg_MAE  = round(mean(roll_results$MAE), 3)
)
kable(avg_df, caption = "Average Rolling-Origin Accuracy")
```

Table 10: Average Rolling-Origin Accuracy

Avg_RMSE	Avg_MAE
2.29	1.916

The rolling origin backtest confirms that with the exception of origin 90, the error metrics of the model are relatively stable with slight variations which can be attributed to decreased sample sizes.

Interestingly, origin 90 (which corresponds to mid-2022) reveals the spike in the model's errors. This is a shock not previously captured, which can be attributed to the real-time event of the Fed's aggressive rate hiking cycle. This uncaptured shock negatively affects ARIMA models which are linear models and likely led to the model uncertainty, increasing the prediction interval.

Conclusion

In all, we were able to conduct a full scale time series analysis using Box-Jenkins methodology to forecast Nominal Broad Dollar Index values. We accomplished this by differencing our data to address non-stationarity, performing model selection using AICc, conducting in-depth residual analysis, forecasting with prediction interval and applying a rolling origin backtest for model evaluation. The ARIMA(2,1,2) model

selected had the lowest AICc of all model candidates and most closely resembled Gaussian white noise. This model had strong predictive accuracy which was seen when compared to the validation set.

Special thank you to Dr. Raya Feldman for her assistance throughout this project!

References

https://fred.stlouisfed.org/series/DTWEXBGS